計画行政学会 GIS 分科会 可視化セミナー

WebGISはじめの一歩、自分でコントロールする可視化基盤

アジェンダ

- Git / Githubの説明
- Webプログラミング入門
- ソースコードの公開とは？

上記の内容(前回)を踏まえて、今回は、

WebGISはじめの一歩、自分でコントロールする可視化基盤

についてお話しします。

~準備~

必要なもの

テキストエディタ（メモ帳でもOK、VS Codeがあればベター）
ブラウザ（Chrome/Firefox/Edge など）

ホスティングをするためにサーバーが必要です。
プラグインLive-serverや `http-server` を利用できるようになると簡単です。
(インターネットに成果を公開したい場合はGitHub Pagesでホスティングするとよい)

ファイル作成

デスクトップなどに `handson` フォルダを作成し、その中に `leaflet`
そのなかに、`index.html` を作成してください。

Leaflet編

Step1 地図を表示させる

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Leaflet 基本の地図</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- Leaflet CSS -->
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <style>
        body { margin: 0; padding: 0; }
        #map { width: 100%; height: 100vh; }
    </style>
</head>
<body>
    <div id="map"></div>

    <!-- Leaflet JavaScript -->
    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

    <script>
        // 地図の初期化（東京駅中心）
        const map = L.map('map').setView([35.6812, 139.7671], 13);

        // 背景地図タイルの追加
        L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
            attribution: '&copy; OpenStreetMap contributors'
        }).addTo(map);
    </script>
</body>
</html>
```

以下にアクセス

http://localhost:8080/leaflet/

(使用するホスティングサーバによって、ポート番号が `5000` などであることも)

Step 2: マーカーを追加

</script> の直前に以下を追加：

```
// 東京駅にマーカーを追加
        const marker = L.marker([35.6812, 139.7671]).addTo(map);
        marker.bindPopup('<b>東京駅</b><br>ここが東京の中心です').openPopup();
```

Step 3: 複数のマーカーを追加

```
// 東京駅にマーカーを追加
        const marker = L.marker([35.6812, 139.7671]).addTo(map);
        marker.bindPopup('<b>東京駅</b><br>ここが東京の中心です').openPopup();
```

Step 4: 円を描く

```
// 東京駅から半径5kmの円
L.circle([35.6812, 139.7671], {
    color: 'red',
    fillColor: '#f03',
    fillOpacity: 0.2,
    radius: 5000
}).addTo(map).bindPopup('東京駅から半径5km');
```

Step 5: 背景地図を変更

国土地理院の地図に変更：

```
L.tileLayer('https://cyberjapandata.gsi.go.jp/xyz/std/{z}/{x}/{y}.png', {
            attribution: '&copy; <a href="https://maps.gsi.go.jp/development/ichiran.html">国土地理院</a>'
        }).addTo(map);
```

航空写真に変更：

```
L.tileLayer('https://cyberjapandata.gsi.go.jp/xyz/seamlessphoto/{z}/{x}/{y}.jpg', {
            attribution: '&copy; <a href="https://maps.gsi.go.jp/development/ichiran.html">国土地理院</a>'
        }).addTo(map);
```

チャレンジ

- 自分の住んでいる地域の座標に変更してみよう
- マーカーの色を変えてみよう
- 線を引いてみよう（L.polyline()）

Leafletを使う場合はこちらでもいい(QGISの履修が前提のため)

qgis2web — QGIS Python Plugins Repository
https://plugins.qgis.org/plugins/qgis2web/

- バッファーやボロノイを作る場合などは座標系の変換に注意
  (ユークリッド距離との変換が大変)
- OpenLeyersなども利用できる

Leafletは書き方が変わります！！

https://leafletjs.com/2025/05/18/leaflet-2.0.0-alpha.html

MapLibre編

Step 1: 基本の地図を表示

handson/maplibre/index.html を作成し、以下をコピペ：

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>MapLibre 基本の地図</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- MapLibre CSS -->
    <link rel="stylesheet" href="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.css" />

    <style>
        body { margin: 0; padding: 0; }
        #map { width: 100%; height: 100vh; }
    </style>
</head>
<body>
    <div id="map"></div>

    <!-- MapLibre JavaScript -->
    <script src="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.js"></script>

    <script>
        // 地図の初期化（東京駅中心）
        const map = new maplibregl.Map({
            container: 'map',
            style: 'https://demotiles.maplibre.org/style.json',
            center: [139.7671, 35.6812],
            zoom: 13
        });

        // ナビゲーションコントロール（ズームボタン）を追加
        map.addControl(new maplibregl.NavigationControl());
    </script>
```

Step 2: 国土地理院の地図に変更

MapLibreでは「スタイルJSON」で地図のデザインを定義します。
styleの部分を以下に置き換え：

```
const map = new maplibregl.Map({
            container: 'map',
            style: {
                version: 8,
                sources: {
                    'gsi-std': {
                        type: 'raster',
                        tiles: [
                            'https://cyberjapandata.gsi.go.jp/xyz/std/{z}/{x}/{y}.png'
                        ],
                        tileSize: 256,
                        attribution: '&copy; <a href="https://maps.gsi.go.jp/development/ichiran.html">国土地理院</a>'
                    }
                },
                layers: [
                    {
                        id: 'gsi-std-layer',
                        type: 'raster',
                        source: 'gsi-std',
                        minzoom: 0,
                        maxzoom: 18
                    }
                ]
            },
            center: [139.7671, 35.6812],
```

Step 3: マーカーを追加

ナビゲーションコントロールの後に追加：

```
// 地図が読み込まれたら実行
        map.on('load', function() {
            // 東京の主要駅データ
            const stations = [
                { name: '東京駅', lng: 139.7671, lat: 35.6812 },
                { name: '新宿駅', lng: 139.7006, lat: 35.6896 },
                { name: '渋谷駅', lng: 139.7016, lat: 35.6580 },
                { name: '品川駅', lng: 139.7387, lat: 35.6284 },
                { name: '池袋駅', lng: 139.7109, lat: 35.7295 }
            ];

            // 各駅にマーカーを配置
            stations.forEach(function(station) {
                // ポップアップを作成
                const popup = new maplibregl.Popup({ offset: 25 })
                    .setHTML('<b>' + station.name + '</b>');

                // マーカーを作成
                new maplibregl.Marker()
                    .setLngLat([station.lng, station.lat])
                    .setPopup(popup)
                    .addTo(map);
            });
```

Step 4: 円を描く

```
// 円データ（GeoJSON形式）
            map.addSource('tokyo-circle', {
                type: 'geojson',
                data: {
                    type: 'Feature',
                    geometry: {
                        type: 'Point',
                        coordinates: [139.7671, 35.6812]
                    }
                }
            });

            // 円のスタイルを追加
            map.addLayer({
                id: 'circle-layer',
                type: 'circle',
                source: 'tokyo-circle',
                paint: {
                    'circle-radius': {
                        stops: [
                            [0, 0],
                            [20, 50000]  // ズームレベルに応じた半径
                        ],
                        base: 2
                    },
                    'circle-color': '#ff3333',
                    'circle-opacity': 0.3,
                    'circle-stroke-width': 2,
                    'circle-stroke-color': '#ff0000'
                }
            });
```

Step 5: 3D建物を表示

```
// 3D建物レイヤーを追加
            map.addLayer({
                id: '3d-buildings',
                source: 'composite',
                'source-layer': 'building',
                filter: ['==', 'extrude', 'true'],
                type: 'fill-extrusion',
                minzoom: 15,
                paint: {
                    'fill-extrusion-color': '#aaa',
                    'fill-extrusion-height': [
                        'interpolate',
                        ['linear'],
                        ['zoom'],
                        15, 0,
                        15.05, ['get', 'height']
                    ],
                    'fill-extrusion-base': [
                        'interpolate',
                        ['linear'],
                        ['zoom'],
                        15, 0,
                        15.05, ['get', 'min_height']
                    ],
                    'fill-extrusion-opacity': 0.6
                }
            });
```

オープンデータ色々

https://nlftp.mlit.go.jp/ksj/gml/datalist/KsjTmplt-N03-2024.html

```
提案1：東京23区の境界データ（ポリゴン）
javascript// 国土数値情報から取得可能
const tokyoWards = 'https://raw.githubusercontent.com/dataofjapan/land/master/tokyo23.geojson';
→ コロプレスマップ（色分け地図）を作成
提案2：東京の鉄道路線（ライン）
javascript// 国土数値情報の鉄道データ
const railwayLines = 'https://nlftp.mlit.go.jp/ksj/gml/data/N02/N02-23/N02-23_GML.zip';
→ 路線図の可視化
提案3：避難所データ（ポイント）
javascript// 国土数値情報の避難所データ
const shelters = 'https://nlftp.mlit.go.jp/ksj/gml/data/P20/P20-12/P20-12_13_GML.zip';
→ ポイントマップ、ヒートマップ
```

Asahinaさんのハンズオン資料がとてもよくできていておすすめ！
https://zenn.dev/asahina820/books/c29592e397a35b

(OSGeoの私の上役ですが...)

DeckGL編

Step 1: 基本の地図を表示

```html
<!doctype html>
<html lang="ja">
<head>
  <meta charset="utf-8" />
  <title>deck.gl + GSI タイル背景サンプル</title>
  <meta name="viewport" content="initial-scale=1,maximum-scale=1,user-scalable=no" />
  <style>
    html, body, #map {
      height: 100%;
      margin: 0;
      padding: 0;
    }
  </style>

  <!-- CSS for maplibre-gl (必要に応じてバージョンを合わせる) -->
  <link href="https://unpkg.com/maplibre-gl@2.4.0/dist/maplibre-gl.css" rel="stylesheet" />
</head>
<body>
  <div id="map"></div>

  <!-- ライブラリ読み込み (バージョンは適宜更新) -->
  <script src="https://unpkg.com/maplibre-gl@2.4.0/dist/maplibre-gl.js"></script>
  <script src="https://unpkg.com/deck.gl@8.10.21/dist.min.js"></script>

  <script>
    // Map container
    const map = new maplibregl.Map({
      container: 'map',
      style: {
        version: 8,
        sources: {},
        layers: []
      },
      center: [139.767125, 35.681236], // 東京駅 (経度, 緯度)
      zoom: 12
    });

    // GSI (地理院地図) のタイルURL (例：ベース地図 (標準地図))
    // タイル仕様: https://maps.gsi.go.jp/development/ichiran.html
    // ここでは仮にして標準地図のタイル URL テンプレートを使用
    const gsiTileUrl = 'https://cyberjapandata.gsi.go.jp/xyz/std/{z}/{x}/{y}.png';

    map.on('load', () => {
      // GSI を raster タイルソースとして追加
      map.addSource('gsi-base', {
        type: 'raster',
        tiles: [gsiTileUrl],
        tileSize: 256,
        attribution: '国土地理院'
      });

      // 背景レイヤ (タイル) を追加 (最背面に表示)
      map.addLayer({
        id: 'gsi-base-layer',
        type: 'raster',
        source: 'gsi-base',
        paint: {}
      }, /* beforeId */ undefined);

      // deck.gl の描画用 Canvas を map に重ねる
      // deck.gl は MapboxStyle 連携ではなく、maplibre の map 容器上に overlay する方法を取る
      const deckCanvas = document.createElement('canvas');
      deckCanvas.id = 'deck-canvas';
      deckCanvas.style.position = 'absolute';
      deckCanvas.style.top = 0;
      deckCanvas.style.left = 0;
      deckCanvas.style.pointerEvents = 'none'; // 地図操作を妨げない
      map.getContainer().appendChild(deckCanvas);

      // Canvas を map に合わせてリサイズするヘルパー
      function resizeCanvas() {
        const rect = map.getContainer().getBoundingClientRect();
        deckCanvas.width = rect.width;
        deckCanvas.height = rect.height;
        deckCanvas.style.width = rect.width + 'px';
        deckCanvas.style.height = rect.height + 'px';
      }
      resizeCanvas();
      window.addEventListener('resize', resizeCanvas);

      // deck.gl のインスタンスを作成 (MapView を使わない単純な例)
      const deckgl = new deck.DeckGL({
        canvas: deckCanvas,
        width: '100%',
        height: '100%',
        initialViewState: {
          longitude: 139.767125,
          latitude: 35.681236,
          zoom: 12,
          pitch: 0,
          bearing: 0
        },
        controller: false, // maplibre 側で操作するため無効化
        layers: [
          // 例: シンプルなScatterplotLayer
          new deck.ScatterplotLayer({
            id: 'scatter',
            data: [
              { position: [139.767125, 35.681236], size: 100, color: [255, 0, 0] }, // 東京駅
              { position: [139.774, 35.6895], size: 80, color: [0, 128, 255] } // 例
            ],
            getPosition: d => d.position,
            getRadius: d => d.size,
            getFillColor: d => d.color,
            radiusUnits: 'pixels'
          })
        ]
      })
      });

      // maplibre の表示変化に合わせて deck.gl を更新
      function syncDeckWithMap() {
        const center = map.getCenter();
        const zoom = map.getZoom();
        const bearing = map.getBearing();
        const pitch = map.getPitch();

        deckgl.setProps({
          viewState: {
            longitude: center.lng,
            latitude: center.lat,
            zoom: zoom - 1, // deck.gl と maplibre のズーム基準差がある場合調整 (必要に応じて)
            bearing: -bearing, // 座標系差があれば調整 (通常はそのまま)
            pitch: pitch
          }
        });
      }

      // 初回同期
      syncDeckWithMap();

      // map の移動時に同期
      map.on('move', () => {
        resizeCanvas();
        syncDeckWithMap();
      });

      // map のズーム中 (スムーズな同期が必要なら追加)
      map.on('zoom', syncDeckWithMap);
      map.on('rotate', syncDeckWithMap);
      map.on('pitch', syncDeckWithMap);
    });
  </script>
</html>
```

Step 2: ポイントデータを表示

主要駅をScatterplotLayerで表示します。
layers: [] の部分を以下に置き換え：

```
layers: [
  new deck.ScatterplotLayer({
      id: 'stations',
      data: [
          { name: '東京駅', coordinates: [139.7671, 35.6812], passengers: 462000 },
          { name: '新宿駅', coordinates: [139.7006, 35.6896], passengers: 775000 },
          { name: '渋谷駅', coordinates: [139.7016, 35.6580], passengers: 379000 },
          { name: '品川駅', coordinates: [139.7387, 35.6284], passengers: 379000 },
          { name: '池袋駅', coordinates: [139.7109, 35.7295], passengers: 558000 }
      ],
      getPosition: d => d.coordinates,
      getRadius: d => Math.sqrt(d.passengers) * 2,
      getFillColor: [255, 140, 0],
      pickable: true,
      radiusMinPixels: 5,
      radiusMaxPixels: 50
  })
]
```

Step 3: ツールチップを追加

ポイントにマウスを乗せると情報を表示します。
new deck.DeckGL({ の設定に以下を追加：

```
            getTooltip: ({object}) => object && {
              html: `<strong>${object.name}</strong><br/>乗降客数: ${object.passengers.toLocaleString()}人/日`,
              style: {
                  backgroundColor: '#333',
                  color: '#fff',
                  padding: '10px',
                  borderRadius: '5px'
              }
          },
```

Step 4: ヒートマップを表示

駅の密集度をヒートマップで可視化します。
layers配列に追加：

```
new deck.HeatmapLayer({
    id: 'heatmap',
    data: [
        { coordinates: [139.7671, 35.6812], weight: 462 },
        { coordinates: [139.7006, 35.6896], weight: 775 },
        { coordinates: [139.7016, 35.6580], weight: 379 },
        { coordinates: [139.7387, 35.6284], weight: 379 },
        { coordinates: [139.7109, 35.7295], weight: 558 },
        { coordinates: [139.7638, 35.6938], weight: 200 },
        { coordinates: [139.7452, 35.6939], weight: 180 },
        { coordinates: [139.6760, 35.6989], weight: 150 }
    ],
    getPosition: d => d.coordinates,
    getWeight: d => d.weight,
    radiusPixels: 60,
    intensity: 1,
```

Step 5: 3D表示（ヘキサゴンレイヤー

データを3Dのヘキサゴン（六角柱）で表示します。

まず、視点を少し傾けます。initialViewState を変更：

```
initialViewState: {
    longitude: 139.7671,
    latitude: 35.6812,
    zoom: 11,
    pitch: 45,  // 傾き
    bearing: 0
},
```

```
                new deck.HexagonLayer({
                  id: 'hexagon',
                  data: [
                      { coordinates: [139.7671, 35.6812], passengers: 462000 },
                      { coordinates: [139.7006, 35.6896], passengers: 775000 },
                      { coordinates: [139.7016, 35.6580], passengers: 379000 },
                      { coordinates: [139.7387, 35.6284], passengers: 379000 },
                      { coordinates: [139.7109, 35.7295], passengers: 558000 },
                      { coordinates: [139.7638, 35.6938], passengers: 200000 },
                      { coordinates: [139.7452, 35.6939], passengers: 180000 },
                      { coordinates: [139.6760, 35.6989], passengers: 150000 }
                  ],
                  getPosition: d => d.coordinates,
                  getElevationWeight: d => d.passengers,
                  elevationScale: 0.01,
                  radius: 500,
                  coverage: 0.8,
                  extruded: true,
                  pickable: true
              }),
```

次回予告

福井県立大学 地域経済研究所
青木和人先生

公共オープンデータ利活用の勘所

国や自治体からはどのようなデータが公開されていますが、どのようなプラットフォームで公開されているのか、あるいはどのような性質のデータが多いのかをあらかじめ知っておけばデータ選定の効率化が行えます。
現時点でどんなデータが出ているかを踏まえて、利活用事例についてお伝えし、
オープンデータが公開されることによる受益者をどう設定するべきか、政策立案にどのように繋げるかの糸口をこれまでの活動を通してお伝えします。

12月13日土曜日 18:00 -
60 ~ 90 分ほどを予定しています。

場所 計画行政学会GIS研究会 Discord (https://discord.gg/aThG5Vge9g)